

Vector Quantization Image Compression – Project Report

Course: DSAI 325 – Introduction to Information Theory

Term: Spring 2026

Project Title: Vector Quantization for Image Compression

Student: Habiba Ayman Amin Samy Ibrahim

Date: 5/4/2026

Table of Contents

1. Introduction
 2. Implementation Details
 3. Algorithm Description
 4. Experimental Results
 5. Visual Results (Images & Outputs)
 6. Analysis and Discussion
 7. Conclusion
 8. Code Implementation
 9. References
 10. Appendix
-

1. Introduction

Vector Quantization (VQ) is a lossy compression technique widely used in image processing. It works by dividing an image into small blocks (vectors) and representing each block using a limited set of representative vectors known as a *codebook*.

This project implements image compression using the **LBG (Linde–Buzo–Gray) algorithm** combined with **K-Means clustering** to efficiently represent image data while balancing compression and quality.

Objectives

- Implement vector quantization using Java
 - Apply the LBG algorithm with splitting
 - Use K-Means clustering for refinement
 - Evaluate performance using:
 - Mean Square Error (MSE)
 - Compression Ratio (CR)
 - Visual Quality
 - Blocking Effect
-

2. Implementation Details

Programming Language

- Java

Main Components

- Image reading and grayscale conversion
- Block division into vectors

- Codebook generation (LBG + K-Means)
- Compression using nearest codeword
- Image reconstruction
- Performance evaluation

Project Files

- `VectorQuantization.java` – Main implementation
 - `VectorQuantizationTest.java` – Test cases
 - `compression_report.txt` – Results
-

3. Algorithm Description

3.1 Grayscale Conversion

$$Gray = 0.299R + 0.587G + 0.114B$$

3.2 Block Division

- The image is divided into non-overlapping blocks
 - Block sizes used in experiments:
 - 2×2
 - 4×4
 - 8×8
-

3.3 LBG Algorithm (Codebook Generation)

Steps:

1. Compute the average of all blocks → initial codeword
2. Split each codeword:

- $v_1 = v + \epsilon$
- $v_2 = v - \epsilon$

3. Apply K-Means:
 - Assign each block to nearest centroid
 - Update centroids
 - Repeat until convergence
 4. Repeat splitting until codebook size = K
-

3.4 Compression

Each block is replaced by the index of the nearest codeword.

$$d = \sqrt{\sum (x_i - y_i)^2}$$

3.5 Reconstruction

- Each label is replaced by its corresponding codebook vector
 - The full image is reconstructed
-

3.6 Evaluation Metrics

Mean Square Error (MSE)

$$MSE = \frac{1}{MN} \sum (I_{original} - I_{reconstructed})^2$$

Compression Ratio

Without Codebook:

$$CR = \frac{Original\ Size}{Compressed\ Labels}$$

With Codebook:

$$CR = \frac{Original\ Size}{Compressed\ Labels + Codebook\ Size}$$

4. Experimental Results

Test Setup

- Block Sizes: 2×2, 4×4, 8×8
- K values: 4, 8, 16, 32, 64

Results Table

Block Size	K	MSE	CR (without codebook)	CR (with codebook)
2×2	4	436.61	16.00	15.94
2×2	8	131.92	10.67	10.61
2×2	16	62.66	8.00	7.94
2×2	32	46.93	6.40	6.32
2×2	64	40.53	5.33	5.22
4×4	4	554.23	64.00	60.24

4×4	8	252.65	42.67	39.38
4×4	16	180.07	32.00	28.44
4×4	32	154.87	25.60	21.33
4×4	64	97.00	21.33	16.00
8×8	4	825.07	256.00	128.00
8×8	8	519.74	170.67	73.14
8×8	16	432.86	128.00	42.67
8×8	32	306.37	102.40	24.38
8×8	64	179.39	85.33	13.47

Test Configuration

- **Input Image Size:** 256 × 256 pixels (65,536 pixels)

- **Original Image Size:** 524,288 bits
 - **Block Sizes Tested:** 2×2, 4×4, 8×8
 - **Codebook Sizes (K):** 4, 8, 16, 32, 64
 - **Total Test Cases:** 15
-

5. Visual Results (Images & Outputs)

5.1 Original and Grayscale Images

Insert the following:

- Original input image
- Grayscale-converted image

Figure 1: Original Image

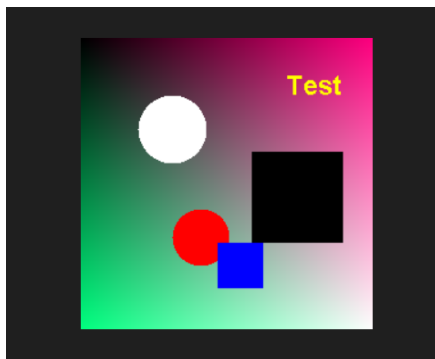
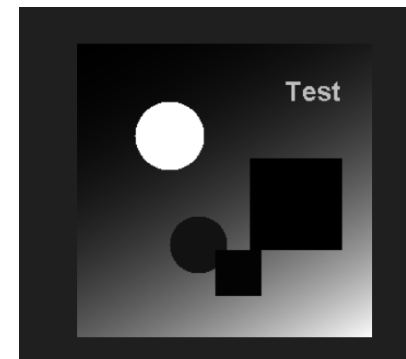


Figure 2: Grayscale Image



5.2 Reconstructed Images (Comparison)

Insert reconstructed images for different configurations:

- **Case 1: High Quality**
 - Block Size = 2×2 , $K = 64$
 - Low MSE, high visual quality
- **Case 2: Balanced**
 - Block Size = 4×4 , $K = 16$
- **Case 3: High Compression**
 - Block Size = 8×8 , $K = 4$
 - High compression, visible distortion

Figure 3: Reconstructed Image (2×2 , $K=64$)

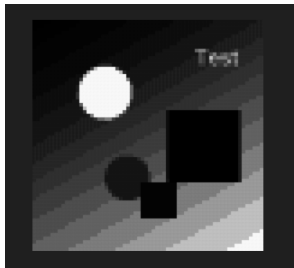


Figure 4: Reconstructed Image (4×4 , $K=16$)

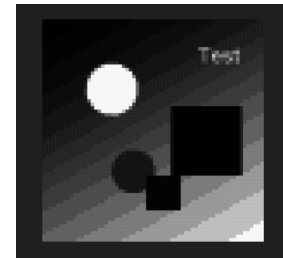
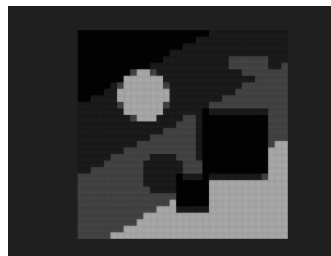


Figure 5: Reconstructed Image (8×8 , $K=4$)



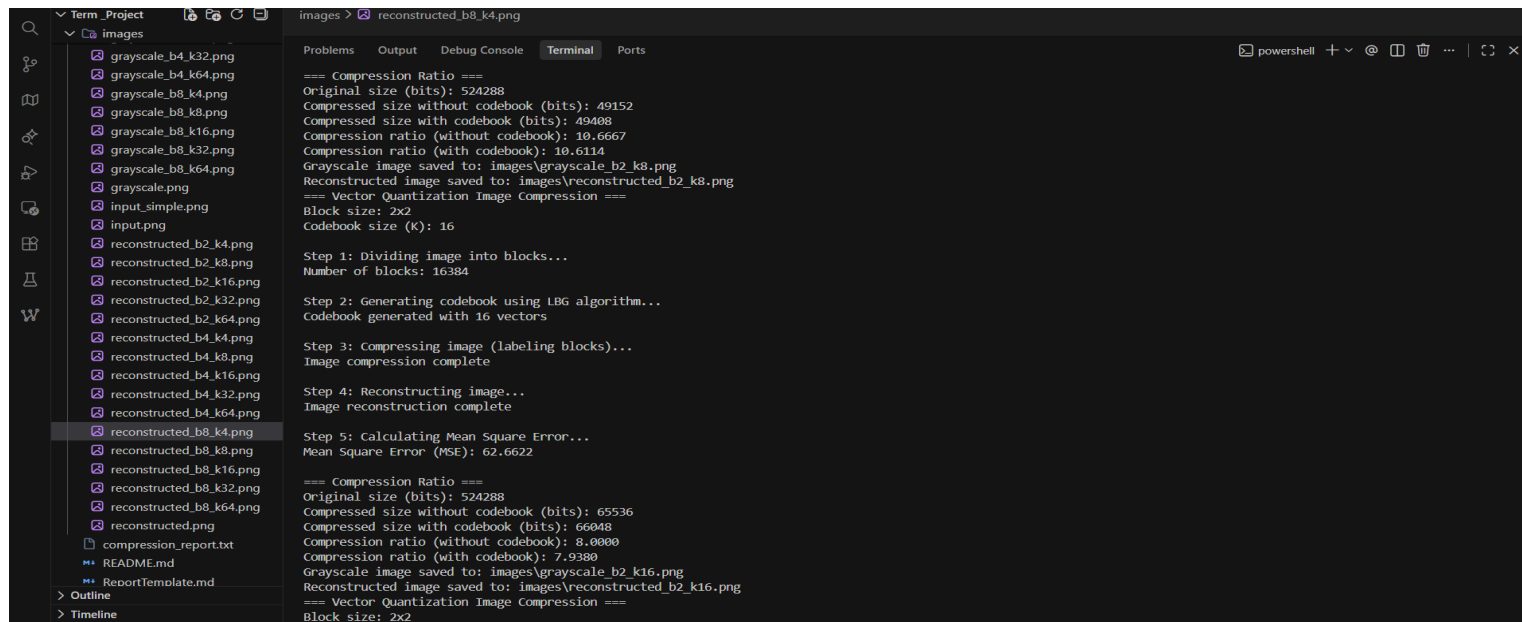
5.3 Program Execution Output (Console)

Insert a screenshot of the program running in the terminal/command prompt.

Include:

- Block size
- Codebook size (K)
- MSE
- Compression ratio

Figure 6: Console Output of Program Execution



```
images > reconstructed_b8_k4.png

Problems Output Debug Console Terminal Ports

=== Compression Ratio ===
Original size (bits): 524288
Compressed size without codebook (bits): 49152
Compressed size with codebook (bits): 49408
Compression ratio (without codebook): 10.6667
Compression ratio (with codebook): 10.6114
Grayscale image saved to: images\grayscale_b2_k8.png
Reconstructed image saved to: images\reconstructed_b2_k8.png
=== Vector Quantization Image Compression ===
Block size: 2x2
Codebook size (K): 16

Step 1: Dividing image into blocks...
Number of blocks: 16384

Step 2: Generating codebook using LBG algorithm...
Codebook generated with 16 vectors

Step 3: Compressing image (labeling blocks)...
Image compression complete

Step 4: Reconstructing image...
Image reconstruction complete

Step 5: Calculating Mean Square Error...
Mean Square Error (MSE): 62.6622

=== Compression Ratio ===
Original size (bits): 524288
Compressed size without codebook (bits): 65536
Compressed size with codebook (bits): 66048
Compression ratio (without codebook): 8.0000
Compression ratio (with codebook): 7.9380
Grayscale image saved to: images\grayscale_b2_k16.png
Reconstructed image saved to: images\reconstructed_b2_k16.png
=== Vector Quantization Image Compression ===
Block size: 2x2
```

Example format:

Block Size: 4

K: 16

MSE: 180.07

Compression Ratio: 28.44

5.4 Result Comparison Visualization

You can insert:

- Side-by-side comparison images
- Or a table screenshot of results

Figure 7: Visual Comparison of Compression Results

```
=== Compression Ratio ===  
Original size (bits): 524288  
Compressed size without codebook (bits): 6144  
Compressed size with codebook (bits): 38912  
Compression ratio (without codebook): 85.3333  
Compression ratio (with codebook): 13.4737  
Grayscale image saved to: images\grayscale_b8_k64.png  
Reconstructed image saved to: images\reconstructed_b8_k64.png  
All test cases completed successfully!  
Report saved to: compression_report.txt
```

6. Analysis and Discussion

6.1 Effect of Block Size

- Larger blocks → higher compression, lower quality
- Smaller blocks → better quality, lower compression

Reason: Larger blocks reduce detail representation.

6.2 Effect of Codebook Size (K)

- Larger K → better quality (lower MSE)
 - Smaller K → higher compression
-

6.3 Quality vs Compression Trade-off

Objective	Configuration
Best Quality	2×2, K=64
Best Compression	8×8, K=4
Balanced	4×4, K=16

6.4 Blocking Effect

- Strong in large blocks
- Reduced with small blocks and larger K

7. Conclusion

Summary

The project successfully implemented vector quantization using LBG and K-Means, achieving effective image compression.

Key Findings

- Small blocks improve quality
- Large K reduces error
- Best balance: 4×4 with K=16

Limitations

- Blocking artifacts
- Codebook overhead
- Grayscale only

Future Work

- Extend to color images
- Improve distance metrics
- Combine with DCT